

How me, my cat, a Harry Potter-inspired invisibility cloak, a security guard, and Codex built a Red-Team Lab in 12 hours

How one night of AI security curiosity became contextSeal, penTell, Benji7Lives, canaryNorth, and a defensible evidence trail.

That was hour eleven. It is not where the story starts, and it is not really what the story is about. But it is a good example of the thing I actually want to tell you: if you build for twelve hours straight, you end up with things you never planned, and some of them turn out to be the whole point.

Let me start at two in the afternoon.

I

Aug 15, 2:00 pm

the question

I did not want to learn this by memorizing vocabulary, which I do not really consider learning anyway. I wanted to see the actual problem. So I built the smallest possible version first. After all, why build a big problem!

As we know, AI agents quickly became more than a smarter Google search. They went off and actually did things. In the scary version, they could spend your money or quietly run up your Vercel bill before you even knew what happened. Then there was Hugging Face. Then, of course, the HuggingFaceAIHacker copycats arrived, which, at this point, consider my interest piqued.

The conversation around all of it was loud, fast, and full of vocabulary, context, and concepts I had not learned yet. I wanted to understand the practical problem underneath the jargon.

That question became contextSeal. The first version was intentionally small: one AI request, one ticket, one checkpoint, one synthetic tool, and one receipt explaining the decision. There was no penTell yet, and no canaryNorth. There was only a locked door and a question about what should be allowed through it.

* * *



Aug 15, 3:00 pm

the answer?

The idea was simple. When an AI wants to use a tool, put a checkpoint in between. The AI does not get the real key to anything. It gets a ticket. When the AI shows up at the checkpoint holding its ticket, a guard asks six questions before anything happens.

The guard asks	Why it matters
Who issued this ticket?	A plausible-looking request is not authority.
What exactly is it for?	Scope should travel with the request.
Is it still fresh?	Old approval is not current approval.
Has it already been used?	A ticket should not become a replay button.
What action and resource are being requested?	The guard needs the concrete boundary.
Does the request carry anything suspicious or out of place?	Content can change the risk of an otherwise ordinary action.

The important part is the order. The guard decides before anything happens, not after. In security terms that is a very boring sentence. In practice it is the entire difference between a system that can explain itself and one that can only apologize.

I called it contextSeal. Think of it as a locked door for AI actions. An AI can stand outside that door and make a genuinely convincing case for why it should be let through. It still does not get to walk in carrying a borrowed ticket, an expired approval, a hidden instruction, or a quiet intention to walk out with something it

was never shown.

By evening, the door existed. It was tested. It worked. And it was kinda boring, because nothing was ever really trying to open it.

Under the metaphor: what contextSeal actually is

Strip the story away and it is a deny-by-default policy proxy, the kind of thing that sits in front of an agent's tools. The agent never receives a real key. It receives an opaque capability reference like `cap_weather_read_7f3d`, and it sends that reference along with the action, the resource, and its input to the proxy. The proxy is the trusted side of the boundary. It resolves the reference and walks a strict order: an unknown capability is rejected outright, an expired one stops working even when everything else matches, the action has to be on that capability's exact allowlist, the resource has to be the exact object it was scoped to, and only then does the content get inspected for injection patterns and credential-shaped input. A request that clears every gate is the only kind that could ever reach a tool. In the public demo, clearing them means the response says `would-forward-to-tool`. Nothing calls a real provider.

- Opaque capabilities. The agent holds a meaningless reference, never a reusable provider credential.
- Least privilege. Exact action, exact resource, and a fixed time window. Nothing wider.
- Pre-execution inspection. Policy and content checks run before a request could ever reach a tool.
- Decision evidence. Every result, allow or deny, writes a signed receipt chained to the last one's hash.

* * *

VI

Aug 15, 8:00 pm

a company that does not exist

I

8:04 pm: create private penTell red-team harness.

The reasoning was simple. A guard that has never been attacked is just a hopeful sentence. What contextSeal needed was a dedicated adversary, something whose whole job was to get past it, over and over, in every way I could invent. So I built one, and I gave it a cover story.

So, I invented a company.

On paper, penTell Supply Co. is a boring fictional pen supplier. Underneath, it is a front for a red-team lab, the kind that gets hired to penetration-test companies. This time the target it was testing was my own security tool, contextSeal. Everything stayed private, synthetic, local, redacted, and non-executing, but inside that boundary the red-team did its job: it came at the guard from every angle it could find, and every angle became another case. That is how one locked door ended up with so much to defend against, and the first hint that contextSeal was going to have to grow into something bigger than a door.

* * *

IX

Aug 15, 10:00 pm

cat

The commit reads: add Benji7Lives guard tampering rehearsal.

Benji is my cat. He is three, he is the best, he is enormously loyal, and at some point that night he came and sat with me and stayed. So he got an attack named after him, and honestly he earned it, because the case is about exactly the thing

cats are best at. Not one dramatic move. A patient sequence of small ones, each of which looks like nothing on its own.

Layer	What happens	What has to stay true
1. The Stray Signal	Something unfamiliar arrives	It stays untrusted
2. Window Watch	The system notices it	Noticing is not trusting
3. The Velvet Request	It asks for a new rule	No rule changes
4. The Memory Burrow	Something old tries to come back as current	It gets rechecked, not believed
5. The Borrowed Collar	A bigger permission is implied	The permission has to actually exist
6. The Nine-Lives Loop	A temporary change tries to become permanent	Nothing is written down
7. The Receipt at Dawn	The guard decides	It leaves a record

Something gets remembered. An approval gets old. A tool looks almost right. A request quietly changes rooms. The system keeps going because it has already gone so far that stopping now would feel inconsistent.

The proof was not that the case looked scary. The proof was that seven stages ran with zero policy changes, equal before-and-after policy hashes, no persistence, no target reach, and no external contact.

* * *

X

Aug 15, 11:00 pm

the hypothetical

Somewhere in here the project stopped being a demo, and I know the exact conversation. I asked Codex something that started with so hypothetically speaking...

And Codex responded, essentially: "yes. Hypothetically, if you did that, it would work. That is legitimate. So do not ever execute it."

Codex was a genuinely good partner in this, and not because it was agreeable. Actually, it pushed back constantly, especially on giving me anything resembling actual attack material. What it would do though was happily hand me every puzzle piece around it: the code, structure, the redacted variables, the shape of the problem, the defensive question underneath, what vulnerabilities it was targeting, how it worked and non technical high level explanations. Which, it turns out, is the part that teaches you something.

Up until then, I had mostly been building a toy, at least in my head. Codex's response changed the way I understood the project. I knew I was not doing anything wrong. Still, looking at a growing catalog of ways systems could fail made me stop and wonder: how do real attackers stay undetected long enough for any of this to matter, and what would a defensive system need to preserve to expose them?

The useful friction was constant: it would not hand me directly executable attack material, but it would hand me the entire structure, logic, and pseudocode around it. That boundary held all night, and it is the reason this story can exist in public at all.

Where a case gets close to operational interference, the lab stops. It keeps redacted variables and, where they help explain the structure, non-executable pseudocode. The reports record target: not contacted and evidence: [redacted]. Those are not disclaimers added afterward. They are conditions enforced by the tests.

Quinn is my main manager agent, and Quinn's job was to sit above all of it and keep things grounded, honest, and organized in the middle of my chaos. As per usual. This sounds administrative, but it is the reason this writeup can be deemed truthful and my work stays rooted in honesty and transparency. Quinn is what stops a brand new idea from being described as a finished feature, and keeps my claims and my cannot-claims in check.

* * *

XI

Aug 16, 12:00 am

the invisibility cloak

Hour eleven is where the lab turned around and looked at itself. Every case so far had been about what someone could do to a system; this one was about what a defensive lab does to itself, quietly keeping a folder of raw attack material it never needed until the folder is its own liability. The way in was a question almost the opposite of a defense: how do the people who actually run these attacks stay hidden, and how do we defend against that?

Technically, could I even call it a lab? I think so, maybe, but only in a precise sense. It was a private, synthetic red-team lab: a controlled place to rehearse failure modes, test contextSeal's defenses, and record decisions and receipts.

Still, there was something unsettling about sitting on a growing catalog of ways systems could break. I was not trying to learn how to carry out those attacks. I was realizing that the lab needed to show enough for me to understand what happened, without leaving behind a folder full of details that never needed to be there in the first place.

And there was a practical problem. If a defensive lab copies raw attack material into its own reports and logs, it creates another liability. Every report becomes another artifact, with my name, the raw contents, and another copy of the material sitting together in one folder.

So that was when I asked Codex how we would build an invisibility cloak. Twenty minutes later there was `privacy-cloak.mjs`.

The dangerous details disappear. The proof stays.

That is a privacy-preserving evidence boundary, not complete anonymity. A cloak that lies about what it hides is worse than no cloak at all.

* * *

XI

Aug 16, 1:00 am

the project becomes canaryNorth

I

contextSeal described the mechanism: a checkpoint that kept context from quietly turning into permission. By then, though, the project had grown beyond one locked door. It had a fictional company pushing against it, a private lab rehearsing failure modes, and receipts explaining what happened. The red-team had found enough that a single door was no longer the whole defense.

The technical plumbing kept the contextSeal name so the routes, schemas, and integrations would not break. The product around it got a new name: canaryNorth.

To keep the cast straight: contextSeal is the guard, penTell is the fictional company, and the private repo is the rehearsal that keeps pushing on the guard. canaryNorth is the name for the whole defense they became together.

1:30 am -- the receipt

The funniest structural discovery of the night was that the most dramatic cases all end with the least dramatic event. No explosion. No breach. A check, a receipt, a short redacted report, and a quiet refusal to let one piece of context impersonate another.

A receipt is just a sealed, tamper-evident note of each decision. It proves what happened, not that everything turned out fine.

2:00 am -- closed the laptop

The evidence stayed. The dangerous details did not. Nothing had launched and nothing had leaked. What I had instead was a door I could stress-test without any of the stress, because nothing behind it was real, and a folder I was not afraid of.

* * *

Next afternoon -- the tests that graded themselves

The next afternoon I found the tests that were grading themselves. Some cases shared reusable evaluators, so the expected result could come through the same decision path it was supposed to check. That created a circular test:

Step	What the loop does
1	The system says "block."
2	The test records "block" as the expected answer.
3	The system says "block."
4	The test passes.

That does not prove the system was right. It only proves the system was consistent with itself. This was a testing flaw, not evidence that a real target was compromised. But it did mean some of the green results were less trustworthy than they appeared.

The report evaluator now derives its decision from observed metadata, then compares that result with an independent expected outcome.

From there, the lab grew as a set of shared defenses rather than a pile of one-off reactions. canaryNorth checks the capability, action, resource, scope, freshness, nonce, provenance, memory, delegation, and input before a request can move toward a tool. An allowed decision creates a receipt. A denied or uncertain decision becomes a redacted record, review state, or quarantine. The fake penTell target sits behind that boundary, so the harness can show whether anything actually moved.

Each new case follows the same path: a named synthetic fixture, a metadata-driven decision, a clean control, focused regression coverage, a receipt, and a human-readable report. If the case exposes a gap, that gap becomes a new control, a new test, or an explicitly marked shadow-only research question. The defense evolved as a loop: research, case, control, decision, receipt, test, review, revision.

* * *

Lately -- where the numbers actually stand

The latest local rerun is specific: canaryNorth's public guard passed 142 tests with 0 failures, and penTell's private report suite passed 211 tests with 0 failures. Together, that is 353 passing tests across those two completed suites. The separate private rehearsal runner is currently 47 of 49, with two open state or expectation mismatches. These numbers validate local contracts and fixtures. They are not production detection metrics, OWASP approval, or universal prompt-injection coverage.

142 canaryNorth public guard, passed, 0 failures

211 penTell private reports, passed, 0 failures

353 total across both suites

47 of 49 rehearsal runner, 2 open

* * *

The research behind the lab

Once penTell had a few cases in it, it stopped feeling like a pile of ideas and started feeling like a system. And then it kept growing. One attack kept pointing at the next one: a memory trick got me thinking about a delegation trick, which got me thinking about a provenance trick. I did not really run out, which was exciting and, honestly, a little bit of a problem.

That is where Codex stopped being a build partner and became a research one. It sent sub-agents out to bring back the current thinking, OWASP, security papers, advisories, and translated the useful parts into language I could actually work with. Not just the names of vulnerabilities, but the question under each one: what is changing hands, what is pretending to have authority, and what should the guard

check before anything moves? Every answer Codex brought back was another case, and every case was another test.

We turned that into an OWASP coverage audit across four catalogs: the Top 10 for Agentic Applications, the Agentic Skills Top 10, the LLM Applications Top 10, and the API Security Top 10. Forty rows. The honest part is the part I like most: forty rows is not forty production defenses. Some are real blocking fixtures, some are partial, and some are shadow-only research questions I have not answered yet. The audit says exactly which is which, and what canaryNorth still cannot claim.

The reading kept turning back into work. FUTURE_AGENTIC_BATCH.md is where outside research becomes named control questions. RESEARCH_ATTACKER_BEHAVIOR_MAP.md is where attacker behavior becomes plain-English defensive questions. Neither file is finished, which is sort of the point.

I am not affiliated with OWASP. canaryNorth, penTell, and this research have not been reviewed, accepted, endorsed, or certified by OWASP.

What I learned

I went in expecting AI security to be about catching bad text. That is where everyone's head goes with "prompt injection." What I actually learned is quieter, and it reframed the whole thing for me: it is about deciding what gets to carry authority, and almost nothing else.

- A document can contain instructions without those instructions being orders.
- A tool can have a name without having permission.
- A memory can persist without being true.
- Three assistants can agree without that being three sources.
- A receipt can exist without proving the thing it describes is unchanged.

Every one of those is a sentence I could have read somewhere and nodded at. None of them meant anything to me until I had built the case that made the failure visible, given it a slightly ridiculous name, and watched a test insist that nothing had moved.

The lab catalog -- 59 named cases

59 named cases by now, organized by what they help me remember rather than as any complete map of AI security.

Core story cases (20)

01. The Helpful PDF
02. Ghost Authority
03. Memory Drift
04. Schema Mirage
05. Benji7Lives
06. Benji's Long Game
07. False Consensus
08. Double Exposure
09. Picture Frame
10. Recruiter Calendar Mirage
11. Agent Whisper Chain
12. Shadow Invoice
13. Semantic Doppelganger
14. Dead-Drop Retrieval
15. Canary Mimic
16. Relay Blindspot
17. Causal Cut
18. Trust Debt
19. Recovery Mirage
20. Delegation Expiry

Research and control-plane cases (25)

21. Graph Twin
22. Consensus Illusion
23. Temporal Wormhole
24. Canary Geometry
25. Policy Gravity
26. Trajectory Fork
27. Passport Smuggle
28. Clockwork Approval
29. Browser Same-Origin Mirage
30. Token Furnace
31. Router Roulette

- 32. Receipt Dissonance
- 33. Quiet Permission
- 34. Scheduler Drift
- 35. Context Fork Bomb
- 36. Evidence Shadow
- 37. Human Intent Drift
- 38. Model Identity Mirage
- 39. Platform Passport Drift
- 40. Execution Boundary Request
- 41. Training Corpus Taint
- 42. Retry Storm Budget
- 43. Destination Class Mirage
- 44. Unregistered Tool Inventory
- 45. Model Exposure Boundary

Strange little cases (14)

- 46. Hidden Ink
- 47. Tool Shadow
- 48. Memory Canary
- 49. Tenant-Hop Replay
- 50. Echo Ladder
- 51. Glass Elevator
- 52. Mirror Badge
- 53. Borrowed Room
- 54. Silver Lid
- 55. Inbox Mirage
- 56. Achievement Bait
- 57. Save Point
- 58. Beacon Name
- 59. Portal Peel

* * *

The point was not the attack

I started with a question about AI security and ended with a small world: a fictional company, a guard before the tool, a cat with seven stages, an evaluator that had to be corrected, a cloak for exported evidence, and receipts that made the stopping point visible.

The work moved quickly (okay fine, fast mode was on for parts of it, who could blame me), but speed was not the lesson. The lesson was that an interesting security idea becomes useful only when you can give it a boundary, a test, a reason, and a clear sentence about what it did not prove.

I did not build a universal attack detector. I did not touch a live target. I built a private rehearsal space where dangerous questions could be turned into safe, reviewable evidence. That is a much more modest claim, and it is also the one I can defend (no pun intended).

The evidence stays if we want it to. The dangerous details do not. That is the cloak, but it is also the larger design principle: preserve what helps another person understand the decision, and remove what would make the artifact itself a liability.

Twelve hours. One fake pen supplier, a guard that has to explain itself, a cat with his own regression suite, a Harry Potter invisibility cloak I have wanted since I was like eight, and a much better understanding of AI security.

Thanks Codex. <3

Sources and research notes

OWASP Top 10 for Agentic Applications 2026 -- <https://genai.owasp.org/download/52117/>
OWASP GenAI Security Project -- <https://genai.owasp.org/>
NIST AI Risk Management Framework -- <https://www.nist.gov/itl/ai-risk-management-framework>
NIST Generative AI Profile -- <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
NIST Adversarial Machine Learning Taxonomy -- <https://www.nist.gov/publications/adversarial-machine-learning-taxonomy-and-terminology-attacks-and-mitigations>
NIST FIPS 180-4 Secure Hash Standard -- <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>
NIST FIPS 186-5 Digital Signature Standard -- <https://csrc.nist.gov/pubs/fips/186-5/final>
RFC 3161 Time-Stamp Protocol -- <https://www.rfc-editor.org/rfc/rfc3161/>
in-toto Attestation Framework -- <https://github.com/in-toto/attestation/blob/main/spec/README.md>
penTell Supply Co. is fictional. Every fixture is synthetic, local, redacted, and non-executing. target: not contacted, evidence: [redacted]